

Computer Vision Practical Report:

DILATION TECHNIQUE

1. AIM

To expand foreground object boundaries and fill small holes/gaps in a binary/grayscale image using a structuring element (kernel) with `cv2.dilate()` in OpenCV, display the original and dilated images, and save the result.

2. PROCEDURE

- Load the input image using `cv2.imread()` and convert it to grayscale or a binary mask using `cv2.threshold()`.
- Define a spatial structuring element (kernel) using `cv2.getStructuringElement()` (e.g., rectangular, elliptical, or cross-shaped).
- Apply the dilation operation using `cv2.dilate()`, specifying the kernel size and iteration count.
- Display the original and dilated images side-by-side using Matplotlib.
- Save the output image to disk using `cv2.imwrite()`.

3. PROGRAM CODE

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image
img_bgr = cv2.imread('E30-IMG.png')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# 2. Define Structuring Element (5x5 kernel)
kernel = np.ones((5, 5), np.uint8)
```

3. Perform Dilation

```
dilated = cv2.dilate(img_rgb, kernel, iterations=1)
```

4. Display side-by-side on a SINGLE PAGE

```
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
```

```
axes[0].imshow(img_rgb)
```

```
axes[0].set_title('Original Image')
```

```
axes[0].axis('off')
```

```
axes[1].imshow(dilated)
```

```
axes[1].set_title('Dilated Image')
```

```
axes[1].axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

4. OUTPUT

Original Image



Dilated Image



5. RESULT

The Python script executed successfully. The input image was loaded.